PORTFOLIO

손지혁

백엔드 개발자

빠르게 배우고 바로 적용하는 백엔드 개발자입니다.

이메일	jhson0507@gmail.com
GitHub	https://github.com/SonJi0507
사이트	https://sonji0507.github.io

01 · 사이드 프로젝트 · 운영 중

OurBibleNote

성경 구절 간의 관계를 그래프로 시각화하고 QT 노트를 연결하는 웹 서비스입니다. 구약과 신약을 한 장의 지도로 보고, 연결된 장을 따라 읽으며 그룹과 묵상 노트를 남깁니다.

문제

성경은 구절이 서로를 가리키는데, 그 관계를 한 장의 지도로 보면서 묵상 노트까지 같은 자리에 두기가 어려웠습니다. 역본을 바꿔도 같은 자리가 이어져야 했습니다.

역할

혼자 기획부터 개발, 배포, 운영까지 맡았습니다.

기술

Django · DRF · Python · Next.js · React · TypeScript · PostgreSQL · Redis · Docker · GitHub Actions · Playwright

기술적으로 손댄 부분

- Django 6 + DRF 백엔드(Python 3.14, uv)와 Next.js 15(App Router, React 19, TypeScript) 프론트엔드를 한 모노레포로 구성했습니다.
- 성경 4개 역본(개역개정·개역한글·NIV·NLT)을 장 단위 그래프 노드로 구성하고, 노드·엣지·태그·그룹 공유 API를 설계했습니다.
- Google·Kakao·Naver 소셜 로그인은 state 검증과 Django 세션 쿠키로 인증합니다.
- 데이터 시딩은 멱등합니다. Excel에서 Bible 테이블을 거쳐 그래프 노드까지 이어집니다.
- GitHub Actions에서 ruff·ty·마이그레이션 누락 검사·pytest(PostgreSQL)와 프론트 lint·tsc·Vitest·Playwright E2E를 돌립니다. E2E는 CSP 위반 0건을 확인합니다.
- 홈서버에 Docker Compose로 운영합니다. PostgreSQL 18, Redis, 리버스 프록시, nonce 기반 CSP를 씁니다.
- 에이전트용 가이드를 두고, lint·타입 검사·테스트·E2E를 CI로 강제해 도구가 작성한 코드도 같은 기준으로 검증합니다.

결과

- https://ourbiblenote.com 에서 운영 중입니다.
- 배포 전에 lint, 타입 검사, pytest, E2E가 통과해야 합니다.

링크

- ourbiblenote.com

02 · 개인 인프라 · 진행 중

홈서버

집 Mac Mini로 개인 인프라와 배포를 직접 돌리는 작업입니다. 클라우드 콘솔을 흉내 내는 대신, 앱을 올렸다 내리고 저장소를 나누고 공개 면만 밖으로 냅니다.

문제

데이터베이스, 캐시, 파일 저장소, 프록시처럼 오래 남는 것과 자주 갈아끼우는 웹앱의 수명이 다릅니다. 데이터는 잘 안 건드리고 앱만 자주 바꾸고 싶었습니다.

역할

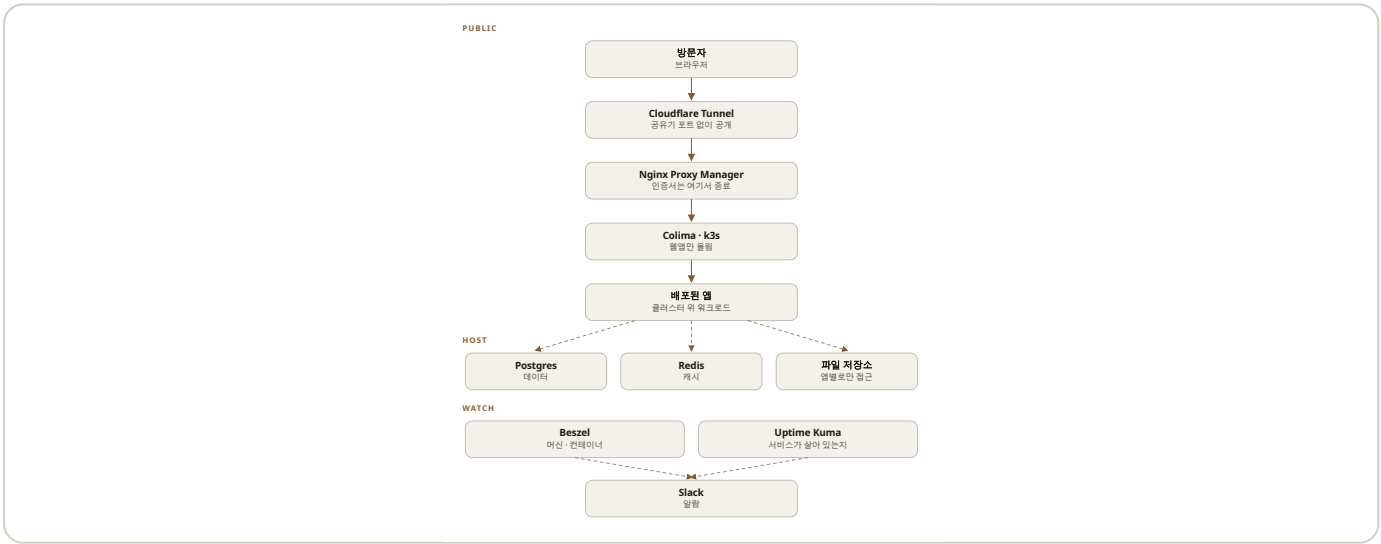
개인 인프라를 직접 구성하고 운영합니다. 인프라 저장소는 공개하지 않습니다.

기술

PostgreSQL · Redis · Docker · Colima · k3s · Cloudflare · Nginx Proxy Manager · Beszel · Uptime Kuma

기술적으로 손댄 부분

- 오래 남는 것은 호스트 위 컨테이너로 두고, 웹앱은 Colima로 띄운 k3s에 올립니다.
- 앱 저장소는 컨테이너 이미지만 만들고, 인프라 저장소가 그 이미지를 받아 올립니다.
- 파일 저장소는 앱마다 접근 권한을 나눕니다. 관리자 계정은 앱에 넣지 않고, 관리 화면은 인터넷에 열지 않습니다.
- 밖에 내보내는 앱은 공유기 포트를 열지 않고 Cloudflare Tunnel로 나갑니다. 인증서는 Nginx Proxy Manager에서 끊습니다.
- 머신과 컨테이너 상태는 Beszel로, 서비스가 살아 있는지는 Uptime Kuma로 봅니다. 대시보드는 밖에 열지 않습니다.



결과

- OurBibleNote를 이 구성 위에서 운영합니다.
- 공개 면과 관리 면을 나눠, 대시보드와 관리 화면은 인터넷에 열지 않습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

03 · 미피아 · 2025.10 – 2026.02, 이후 운영

음원 분석 API 플랫폼

외부 개발자용 음원 분석 API 플랫폼을 프로토타입에서 프로덕션으로 다시 설계하고 구축한 일입니다. 2025년 12월 16일 API v1.0을 공개한 뒤 같은 구조 위에서 버전을 늘리며 운영합니다.

문제

밖으로 내놓은 음원 분석 API가 프로토타입에 머물러, 외부 개발자가 그대로 붙이기 어려웠습니다.

역할

백엔드 개발자로 아키텍처 설계와 구축을 맡았습니다. 사실상 단독으로 진행했습니다.

기술

Python SQL

기술적으로 손댄 부분

- API v1.0은 AI 음악 판별(lite·standard·pro), 음악 표절 탐지, 가사 표절 분석, 데이터셋 관리, Webhook 연동을 제공합니다.
- v1.1에서 가사 추출 API를 같은 구조 위에 올렸습니다 (2026.02).
- v1.1.1은 음원을 먼저 올립니다. POST /api/v1/music 이후 musicId로 분석합니다.
- 음원 fingerprint를 서버에 구현해 기능을 확장했습니다.

결과

- 2025.12.16 API v1.0을 공개했습니다.
- 이후 가사 추출, 선업로드, fingerprint까지 같은 플랫폼에서 운영합니다.
- B2B 과금 체계(Credit Wallet) 전환도 이 플랫폼 위에서 이어집니다.

링크

- docs.mippia.com

04 · 미피아 · 2026.08 – 2026.09

B2B Credit Wallet

B2B 플랫폼의 결제·과금 체계를 Credit Wallet으로 전환한 일입니다. 계약 형태가 달라도 같은 파이프라인으로 처리합니다.

문제

선불, 후불, 단건, 정기, 자동 충전처럼 B2B 계약 형태가 나뉘어 있어 과금 경로가 갈라졌습니다.

역할

백엔드 개발자로 전환을 맡았습니다.

기술

Python

기술적으로 손댄 부분

- 선불·후불·단건·정기·자동 충전을 하나의 과금 파이프라인으로 대응하도록 구성했습니다.

결과

- 계약 형태가 달라도 하나의 파이프라인으로 대응할 수 있게 됐습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

05 · 미피아 · 2025.08 – 진행중

Observability

메트릭·로그·트레이스가 없던 환경에 관측 체계를 처음 구축한 일입니다. 사람이 찾던 장애를 서비스별 Slack 알림으로 바꿨습니다.

문제

장애를 사람이 직접 발견해야 했고, 원인을 따라갈 관측 자료가 없었습니다.

역할

백엔드 개발자로 관측 체계를 도입하고 운영합니다.

기술

OpenTelemetry · Prometheus · Grafana · Loki · Tempo · Slack

기술적으로 손댄 부분

- 메트릭, 로그, 트레이스를 서비스에 붙여 장애 인지와 원인 추적이 가능하게 했습니다.
- 사람이 직접 발견하던 장애를 서비스별 Slack 알림으로 통지합니다.

결과

- 관측이 없던 환경에서 장애를 알림으로 인지하고 원인을 추적할 수 있게 됐습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

06 · 미피아 · 2025.05 – 2026.10

조회·응답 성능

상시로 맡은 성능·운영 개선입니다. 자주 보는 조회와 API 응답의 꼬리를 줄이고, 배포는 CI/CD로 고정했습니다.

문제

주요 테이블 조회와 API 응답이 느렸고, 배포 안정성도 손봐야 했습니다.

역할

백엔드 개발자로 인덱스, 응답 경로, CI/CD를 맡았습니다.

기술

SQL · Python · GitHub Actions

기술적으로 손댄 부분

- 주요 테이블에 인덱스를 추가해 조회 경로를 줄였습니다.
- API 응답의 p95·p99를 같은 기간에 따라 낮췄습니다.
- CI/CD로 배포 절차를 고정해 배포 안정성과 개발 생산성을 높였습니다.

결과

- 주요 테이블 조회 p95 500ms → 약 5ms (100배).
- API 응답 p95 2s → 0.24s (약 8배), p99 2.4s → 0.48s (5배).

링크

공개된 저장소나 제품 링크는 없습니다.

07 · 미피아 · 2026.02 – 2026.04

검사 시스템 재설계

검사 시스템의 아키텍처와 DB 스키마를 다시 설계한 일입니다. 데이터를 음원 중심으로 다루도록 맞춰 다음 서비스가 갈라질 자리를 만들었습니다.

문제

검사 데이터가 음원 중심의 다음 기능과 맞지 않았고, DB 호출이 많았습니다.

역할

백엔드 개발자로 아키텍처와 스키마 재설계를 맡았습니다.

기술

SQL Python

기술적으로 손댄 부분

- 검사 스키마를 음원 중심으로 재구성했습니다.
- 쿼리 가독성과 유지보수성이 떨어지던 호출 구조를 정리했습니다.

결과

- DB 호출 횟수 30% 감소.
- 음원 중심 관리로 서비스 다각화 기반을 마련했습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

08 · 미피아 · 2026.01 – 2026.02

로그인·토큰

로그인과 토큰 구조를 전면 개편한 일입니다. 재발급이 깨지던 경로를 고치고, 인증 방식을 넓혔습니다.

문제

토큰 재발급에 오류가 있었고, 인증 방식이 한정돼 접근이 좁았습니다.

역할

백엔드 개발자로 인증 구조 개편을 맡았습니다.

기술

JWT OAuth Python

기술적으로 손댄 부분

- 토큰 재발급 흐름을 다시 구성해 실패하던 갱신을 해소했습니다.
- 인증 방식을 확장해 들어올 수 있는 경로를 늘렸습니다.

결과

- 토큰 재발급 오류를 해소했습니다.
- 인증 방식 확장으로 접근성을 개선했습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

09 · 미피아 · 2026.07

공통 스토리지 유틸리티

사내 공통 백엔드 유틸리티로 스토리지와 보안 연동을 뺀 일입니다. 서비스마다 반복하던 클라우드 설정과 파일 재업로드를 라이브러리 뒤로 숨겼습니다.

문제

스토리지 연동에 클라우드별 설정이 반복됐고, 서비스 사이에 파일을 다시 올리는 공정이 남아 있었습니다.

역할

백엔드 개발자로 공통 라이브러리를 만들었습니다.

기술

Python GCP NCP AWS

기술적으로 손댄 부분

- 스토리지 연동에 필요하던 클라우드별 설정 작업을 공통 라이브러리로 모았습니다.
- 서비스 간 파편화 때문에 생기던 파일 재업로드 공정을 제거했습니다.

결과

- 사내 개발자가 클라우드별 스토리지 설정을 반복하지 않게 됐습니다.
- 불필요한 파일 재업로드 공정을 없앴습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

10 · 미피아 · 2026.07

1회용 이메일 도메인 수집

가입 어뷰징을 막기 위해 1회용 이메일 도메인 목록을 사람 없이 갱신하는 파이프라인입니다.

문제

1회용 이메일 도메인 목록을 사람이 갱신하면 데이터 관리 공수가 크고, 목록이 금방 낡습니다.

역할

백엔드 개발자로 수집 파이프라인을 만들었습니다.

기술

Python

기술적으로 손댄 부분

- 1회용 이메일 도메인 목록을 스케줄에 따라 자동으로 최신화합니다.

결과

- 사람 손 없이 목록을 갱신해 데이터 관리 공수를 줄였습니다.
- 2026.07–10 동안 15회 자동 실행됐습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

11 · 미피아 · 2026.04

이메일 발송·트래킹

이메일을 보내고, 반응을 추적하고, 그 결과로 타겟팅하는 흐름을 한 체계로 만든 일입니다.

문제

발송 이후의 반응과 타겟팅이 이어지지 않아, 메일을 자동화할 기반이 없었습니다.

역할

백엔드 개발자로 발송·추적 체계를 구축했습니다.

기술

Python Mailjet

기술적으로 손댄 부분

- 발송, 반응 추적, 타겟팅이 한 흐름으로 이어지게 구성했습니다.

결과

- 이메일 자동화를 이어 갈 기반을 마련했습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

12 · 미피아 · 2026.05

Mucheck Demo

Mucheck AI 판별 솔루션의 데모 페이지를 만들고, 큰 파일 업로드가 서버를 누르지 않도록 경로를 고친 일입니다.

문제

대용량 파일 업로드가 느렸고 서버 부하가 컸습니다.

역할

백엔드 개발자로 데모 페이지와 업로드 경로를 맡았습니다.

기술

Python

기술적으로 손댄 부분

- 판별 솔루션을 시험해 볼 데모 페이지를 구축했습니다.
- 데모의 대용량 파일 업로드 속도를 높이고 서버 부하를 낮췄습니다.

결과

- 대용량 파일 업로드 속도를 개선하고 서버 부하를 줄였습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

13 · 미피아 · 2025.05 – 2025.09

비동기·배치

손으로 돌리던 운영 작업을 비동기 처리, 배치, 스케줄러로 옮긴 일입니다.

문제

반복 운영이 수동이었고, 배치가 돌았는지 보기 어려웠습니다.

역할

백엔드 개발자로 비동기 처리와 스케줄러를 도입했습니다.

기술

Python Celery

기술적으로 손댄 부분

- 반복 작업을 비동기·배치로 옮기고 스케줄러에 올렸습니다.
- 배치 작업의 안정성과 가시성을 운영에서 확인할 수 있게 했습니다.

결과

- 수동 운영 부담을 줄였습니다.
- 배치 작업의 안정성과 가시성을 확보했습니다.

링크

공개된 저장소나 제품 링크는 없습니다.

14 · 미피아 · 2025.06 – 2025.07

요금제 개편

요금제를 개편하면서 마케팅 정책이 코드와 배포에 반영되는 속도를 맞춘 일입니다.

문제

마케팅 정책이 바뀌어도 요금 코드와 배포가 그만큼 빨리 따라가지 못했습니다.

역할

백엔드 개발자로 요금제 개편을 맡았습니다.

기술

Python

기술적으로 손댄 부분

- 요금제 변경이 코드에 반영되는 경로를 정리했습니다.

결과

- 마케팅 정책 반영 속도를 개선했습니다.
- 코드 안정성과 배포 신뢰도를 높였습니다.

링크

공개된 저장소나 제품 링크는 없습니다.